Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

Computer Laboratory Report: Molecular Simulation

This computer laboratory is about simulating the behavior of 40 argon atoms in a 2-dimensions box of 4 nm length at a temperature of 300 K for 50 ps.

a) Algorithm to solve Newton's equations of motion

Expression of the position x(t) and velocity v(t) for the algorithm in the script

In the Python script, the position $\vec{x}_i$ and velocity $\vec{v}_i$ of each particle i are expressed as follows:

$$\begin{cases} x_i = x_i + v_{x_i} dt \\ y_i = y_i + v_{y_i} dt \end{cases} \quad (1)$$

$$\begin{cases} v_{x_i} = v_{x_i} + \frac{1}{2m} F_{x_i} dt, & v_{x_0} = 0 \\ v_{y_i} = v_{y_i} + \frac{1}{2m} F_{y_i} dt, & v_{y_0} = 0 \end{cases} \quad (2)$$

, where $\vec{F}_i = \left[F_{x_i}, F_{y_i}\right]^T$ represents the force acting on the particle.

In vector form:

$$\vec{x}_i = \vec{x}_i + \vec{v}_i dt \quad (3)$$

$$\vec{v}_i = \vec{v}_i + \frac{1}{2m} \vec{F}_i dt \quad (4)$$

, where $\vec{x}_i = [x_i, y_i]^T$, $\vec{v}_i = \left[v_{x_i}, v_{y_i}\right]^T$, and $\vec{F}_i = \left[F_{x_i}, F_{y_i}\right]^T$.

Since molecular dynamics simulations evolve in time, the code iteratively updates $\vec{x}_i$ and $\vec{v}_i$ at each step, which ensures efficient computation. This approach allows for real-time tracking of particle motion while minimizing memory usage, making it well-suited for large-scale simulations.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

Expression of the position x(t) and velocity v(t) for the Leap-frog algorithm

Commented [SK1]: Remove IC for positions, as they are randomly generized

In the Leap-Frog algorithm, the position $\vec{x}_i$ and velocity $\vec{v}_i$ of each particle i are updated as follows:

$$\begin{cases} x_{i+1} = x_i + v_{x_{i+1/2}} dt \\ y_{i+1} = y_i + v_{y_{i+1/2}} dt \end{cases} \tag{5}$$

$$\begin{cases} v_{x_{i+1/2}} = v_{x_{i-1/2}} + a_{x_{i+1}} dt = v_{x_{i-1/2}} \dfrac{dt}{m} F_{x_{i+1}}, & v_{x_0} = 0 \\ v_{y_{i+1/2}} = v_{y_{i-1/2}} + a_{y_{i+1}} dt = v_{y_{i-1/2}} \dfrac{dt}{m} F_{y_{i+1}}, & v_{y_0} = 0 \end{cases} \tag{6}$$

, where $\vec{a}_i = \left[a_{x_i}, a_{y_i}\right]^T$ is the acceleration vector and $\vec{F}_i = \left[F_{x_i}, F_{y_i}\right]^T$, the force vector acting on the particle.

In vector form:

$$\vec{x}_{i+1} = \vec{x}_i + \vec{v}_{i+1/2} dt \tag{7}$$

$$\vec{v}_{i+1/2} = \vec{v}_{i-1/2} + \frac{dt}{m} \vec{F}_{i+1} \tag{8}$$

, where $\vec{x}_i = [x_i, y_i]^T$ and $\vec{v}_i = \left[v_{x_i}, v_{y_i}\right]^T$.

Drawbacks of the two algorithms

The first algorithm is an implementation of Euler's first-order method. This method assumes that both velocity and force remain constant over each time step $[t, t + dt]$, i.e., acceleration does not change within this interval. Nevertheless, the local truncation error in each step is proportional to dt^2. However, since the total number of steps in a simulation is T/dt, where T is the total simulation time, the global error is proportional to dt. Consequently, Euler's method is unsuitable for long-time simulations, as larger time steps introduce significant accumulated inaccuracies and instability, while smaller time steps increase computational cost.

In contrast to Euler's first-order method, the Leap-Frog algorithm updates the positions at full-time steps $(t + dt)$ and the velocities at half-time steps $(t + dt/2)$. This staggered update reduces numerical errors and improves numerical stability by minimizing the number of approximated variables stored at once. While the method still depends on the time step for accuracy, its second-order nature results in a global error proportional to dt^2, making it more accurate and efficient for long-term simulations and energy conservation.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

b) Comparison of the simulation with different ε

In Figures 1 and 2, the energy plots illustrate how changing the Lennard-Jones' parameter ε affects the system's energy distribution. As ε increases, the potential energy also increases, indicating a stronger binding interaction and a more stable system. As energy is conserved, this increase in potential energy is compensated by a decrease in kinetic energy, suggesting that the particles are moving slower and have reduced energy of motion. Additionally, despite shifts in kinetic and potential energy, the total energy remains constant at approximately 20 kJ/mol, demonstrating energy conservation within the system. This balance highlights how energy transforms between forms without altering the system's overall energy.

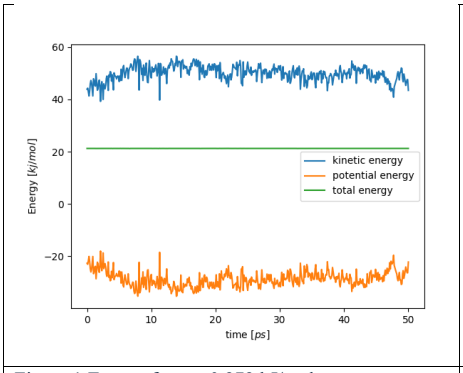


Figure 1:Energy for ε = 0.979 kJ/mol

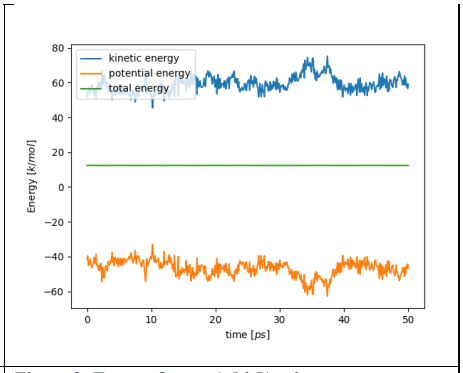


Figure 2: Energy for ε = 1.5 kJ/mol

Figure 3 shows that increasing ε results in a deeper potential well, indicating stronger intermolecular interactions between the argon atoms. This suggests a more stable bonding configuration, where the system exhibits greater structural integrity and is less susceptible to external disturbances (e.g., thermal fluctuations and force changes).

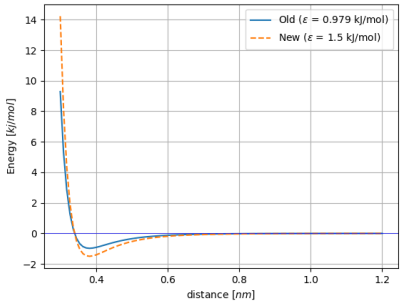


Figure 3: Potential energy as a function of distance

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

c) Effect of time steps larger than 0.01 ps

Observe: in the plots illustrated in Figures 4-17, the number of steps (n) was reduced from 5000 to 500 to reduce running time and computational cost.

Figures 4-9 illustrate the impact of time step dt selection on energy conservation and numerical stability by showing the evolution of the different energies over time for different values of dt.

For $dt = 0.01$ ps and $dt = 0.02$ ps, kinetic and potential energies exhibit small oscillations, while the total energy remains constant, indicating numerical stability. For $dt = 0.05$ ps, the total energy conservation is still maintained. However, minor deviations appear, suggesting proximity to the stability threshold. Beyond $dt = 0.051$, numerical instability emerges, as the total energy starts drifting. For $dt = 0.06$ ps, instability worsens, with fluctuations in kinetic and potential energies becoming pronounced. By $dt = 0.075$ ps, the system is completely unstable, as seen in Figure 9, where the total energy exhibits an unphysical increase, rendering the simulation unreliable. Since numerical instability emerges for $dt = 0.051$, the estimation of the maximum time step is between 0.05 and 0.051 ps.

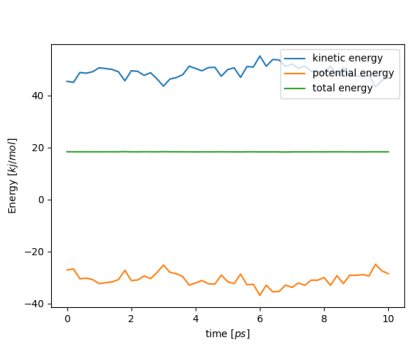


Figure 4: Energies with dt = 0.01 ps

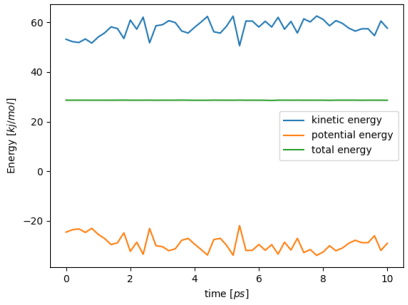


Figure 5: Energies with dt = 0.02 ps

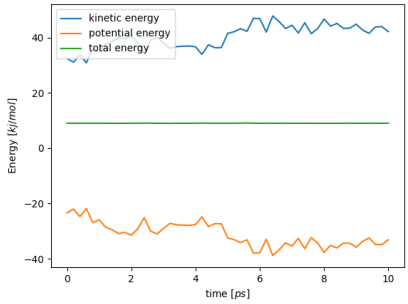


Figure 6: Energies with dt = 0.05 ps

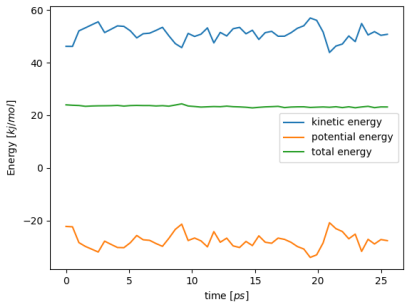


Figure 7: Energies with dt = 0.051 ps

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

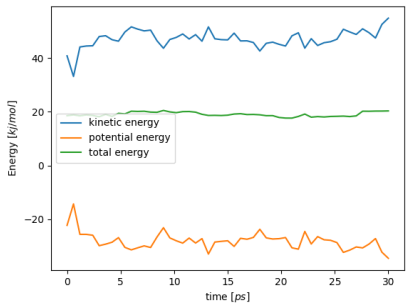


Figure 8: Energies with dt = 0.06 ps

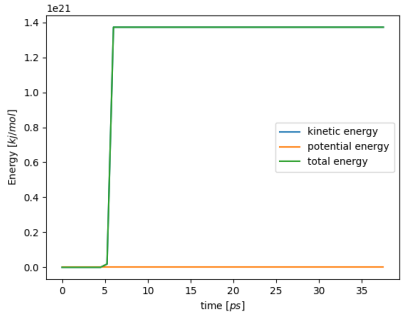


Figure 9: Energies with dt = 0.075 ps

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

d) Effect of mass and temperature on the energies

Simulation with different temperatures for the initial mass n = 40

Figures 10-13 highlight the effect of temperature (T) on energy conservation over time for various temperatures. The number of particles (n) remains constant, ensuring that the system mass does not change.

At lower temperatures ($T = 150$ K, Figure 10), kinetic energy is lower, and potential energy is higher (less negative), indicating weaker intermolecular interactions. As temperature increases ($T = 300, 450, 900$ K, Figures 11-13), kinetic energy rises, while potential energy decreases (becomes more negative), reflecting stronger intermolecular interactions and molecular motion. Ultimately, total energy remains constant, confirming energy conservation in the simulation.

$$KE = \frac{1}{2}mv^2 \tag{9}$$

According to the equipartition theorem, the average kinetic energy per degree of freedom (DOF) is given by:

$$K_{DOF} = \frac{1}{2}k_B T \tag{10}$$

, where k_B is Boltzmann's constant and T is the absolute temperature. This indicates that higher temperatures lead to higher kinetic energy. Since kinetic energy is proportional to the square of the velocity, according to (9), increasing T results in faster molecular motion. This trend is evident in Figures 10-13, where kinetic energy increases with T, while potential energy becomes less negative due to greater molecular separation (i.e., weaker intermolecular bonding).

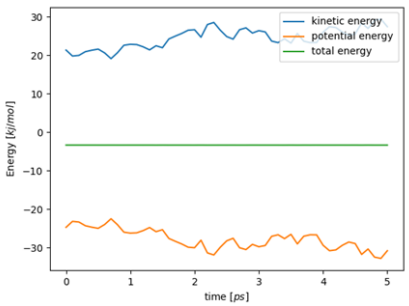


Figure 10: Energies for T = 150 K

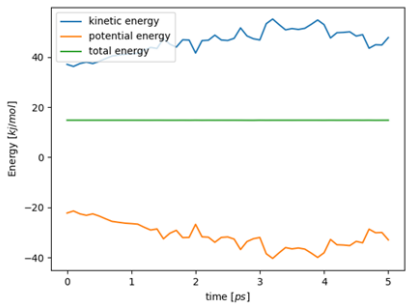


Figure 11: Energies for T = 300 K

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

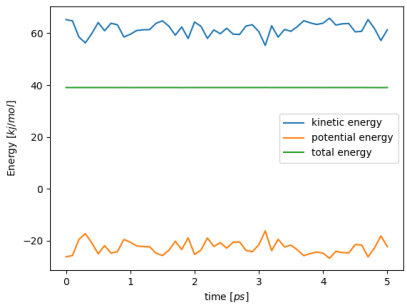


Figure 12: Energies for T = 450 K

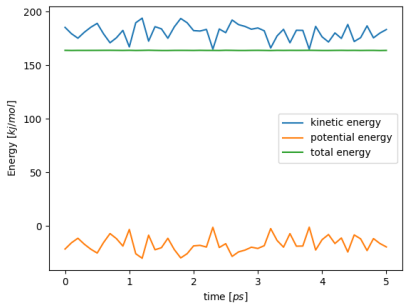


Figure 13: Energies for T = 900 K

Simulation with different masses for temperature T = 300 K

Figures 14-17 underscore the influence of system mass on energy conservation over time for different numbers of particles (n) at a fixed temperature of 300 K. The total mass of the system increases as n increases, allowing an analysis of energy distribution variation with system size.

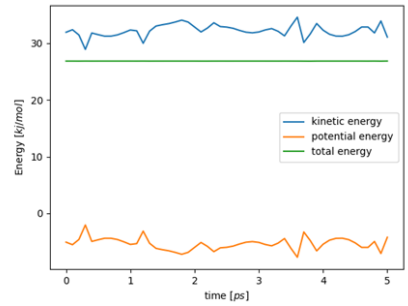


Figure 14: Energies for n = 20

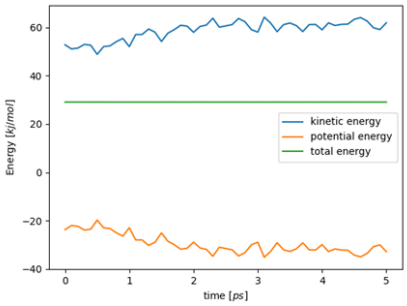


Figure 15: Energies for n = 40

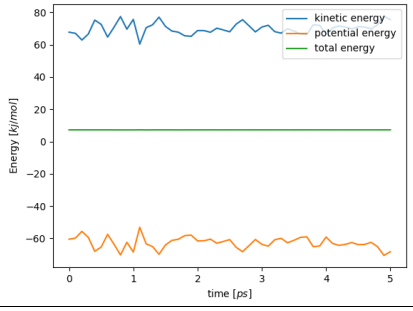


Figure 16: Energies for n = 60

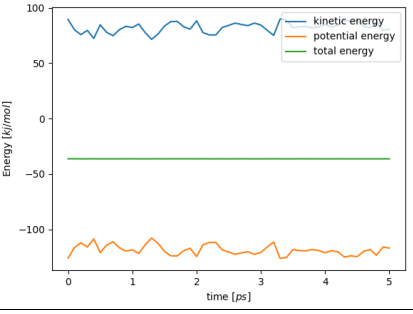


Figure 17: Energies for n = 80

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

From the equipartition theorem, the total kinetic energy follows:

$$K_{particle} = \frac{1}{2}mv^2 \Longrightarrow K_{total} = n \cdot K_{particle} \quad (11)$$

, where each particle has the same average kinetic energy because of the fixed temperature, as shown in (10). As n increases, total kinetic energy rises proportionally, while potential energy becomes more negative, reflecting stronger intermolecular interactions. Despite fluctuations in kinetic and potential energy, total energy remains conserved, confirming stability.

In conclusion, as the time step determines how frequently positions and velocities of the atoms are updated, fewer atoms and higher temperatures require smaller time steps to accurately track motion. Consequently, such simulations require more computational power. Simulations use techniques like bond constraints to increase time steps and improve efficiency.

Yolande Ekpe, aneekpe@kth.se
Sahand Khodabandehloo, sahandkh@kth.se

e) Simulation method choice for the conformation of a small protein

Monte-Carlo method

The Monte-Carlo (MC) method is a stochastic technique that generates molecular conformations through random sampling, relying on energy levels for efficacy—lower energy states are more favorable. This time-independent approach uses random moves and predefined criteria for accepting new conformations, considering energy changes and temperature effects. However, it can be inefficient for larger macromolecules like proteins due to complex energy levels and greater degrees of freedom, making exploration of conformational space challenging. These limitations are important to consider, especially with biomolecules containing around 100,000 atoms.

Molecular dynamics

The Molecular Dynamics (MD) method utilizes Newton's laws of motion to generate various conformations of proteins. It is time-dependent and accurately predicts how protein structures can change in response to different energy states. MD is particularly suited for simulating conformational rearrangements on short-time scales, such as nanoseconds while maintaining atomic-level resolution. This makes it highly relevant for studying the structural conformations of proteins.

Ultimately, the choice between MC and MD—stochastic sampling vs. determinism evolution—depends on the trade-off between computational efficiency and accuracy. Although MC excels in equilibrium sampling, provided sufficient simulation time, it lacks the ability to capture time-dependent properties necessary required to track system evolution. In conclusion, since protein conformational rearrangement is a time-dependent process, MD is the preferred method, as it accurately determines molecular motion using Newton’s equations. Additionally, while a coarse-grained model improves efficiency for large systems, an atomistic model is more suitable for capturing detailed interactions, making it ideal for studying protein dynamics.